CSS

TEIL 2

VERERBUNG (INHERITANCE)

- Eigenschaften werden an Nachfahren vererbt, sofern sie nicht überschrieben werden.

```
body {
    color: blue;
}
```

```
body -> blau
    h1 -> blau
        div -> blau
span -> blau
```

NICHT ALLE WERTE WERDEN VERERBT!

DAUMENREGEL 👍

- 👍 : Alles, was mit Text-Styling zu tun hat (`font-size`, `font-family`, `color`)
- 👎 : Alles, was das Aussehen der Box betrifft (`border`, `width`, `padding`, `background-color`)
- 🔎 Nachschlagen auf MDN Web Docs

DIE KASKADE (CASCADE)

Wenn zwei oder mehr Regeln auf ein Element zutreffen…

Frage: *Welche Regel gewinnt?*

Antwort: *Die Kaskade – der Algorithmus hinter CSS.*

DIE KASKADE

In aufsteigender Wichtigkeit:

1. Quellcode-Reihenfolge (Source order)
2. Spezifität (Specificity)
3. Wichtigkeit (`!important`)

Mehr dazu hier: MDN: Kaskade und Vererbung

QUELLCODE-REIHENFOLGE

- Die Regel, die zuletzt im Stylesheet definiert wurde, gewinnt.

```
.box {
    color: red;
    color: blue; /* Gewinnt! */
}
```

SPEZIFITÄT

- Welcher Selektor ist "genauer"?

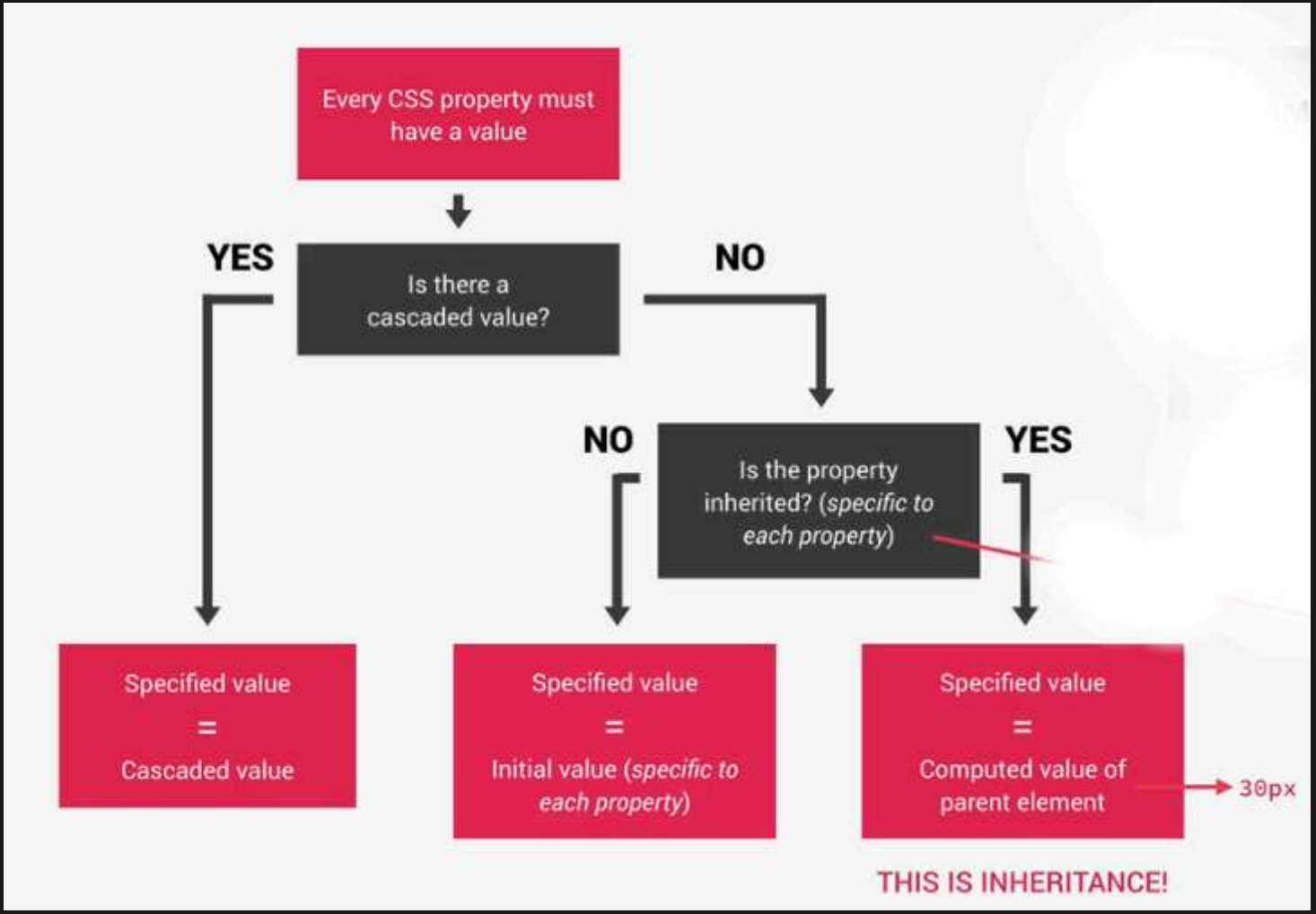
```
Einfache Rangfolge:
    ⇢ * (Universell)
      ⇢ h1 (Element)
        ⇢ .button (Klasse)
          ⇢ #header (ID)
⇢ style="..." (Inline)
```

WICHTIGKEIT (`!important`)

- Damit lässt sich alles andere überschreiben.
- **ABER:** Nutze es nur im Notfall (z.B. Debugging), da es die Wartbarkeit extrem verschlechtert!

```
p {
  color: blue !important;
}
```

KASKADE VS. VERERBUNG?



QUIZ-ZEIT! 💡

FRAGE 1

WELCHE FARBE HAT DAS H1?

```
h1 {
    color: red;
}
h1 {
    color: blue;
}
```

```
<h1>News des Tages</h1>
```

LÖSUNG: BLAU (SOURCE ORDER)

FRAGE 2

WELCHE FARBE HAT DAS H1?

```
.ueberschrift {
    color: red;
}

h1 {
    color: blue;
}
```

```
<h1 class="ueberschrift">News des Tages</h1>
```

LÖSUNG: ROT (KLASSE IST SPEZIFISCHER ALS ELEMENT)

FRAGE 3

WELCHE FARBE HAT DAS H1?

```
#special {
    color: green;
}

.ueberschrift {
    color: red;
}
```

```
<h1 id="special" class="ueberschrift">News des Tages</h1>
```

LÖSUNG: GRÜN (ID IST SPEZIFISCHER ALS KLASSE)

BOX-MODELL: BLOCK VS. INLINE

BLOCK-ELEMENTE: `display: block`

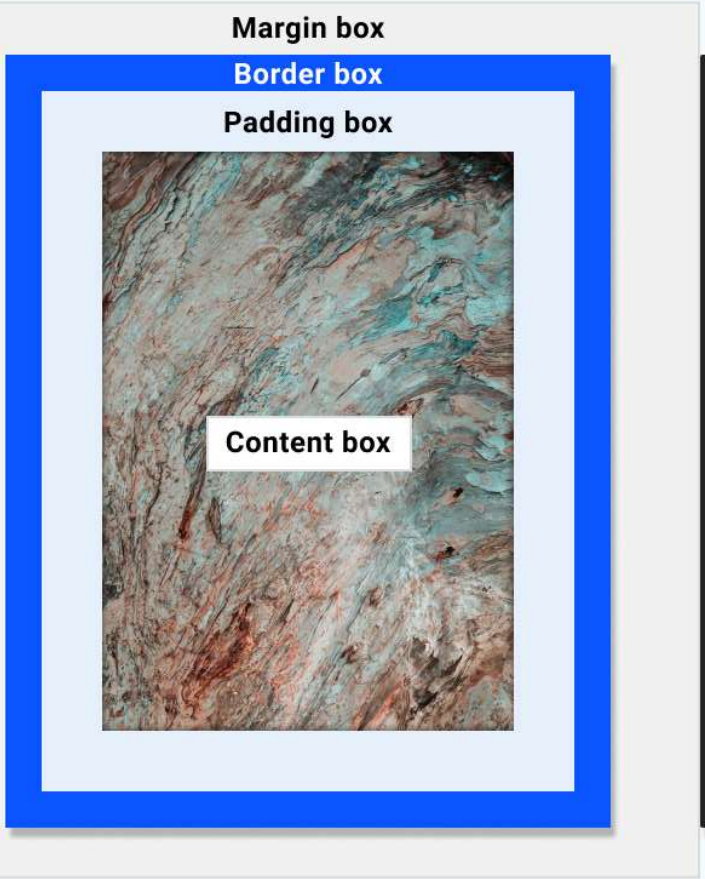
- Nehmen standardmäßig die volle Breite ein
- Beginnen immer in einer neuen Zeile
- Beispiele: `<div>`, `<h1>`, `<p>`, `<ul>`
- Das Box-Modell (Breite, Höhe, Abstände) greift voll

INLINE-ELEMENTE: `display: inline`

- Nehmen nur so viel Platz wie nötig ein
- Brechen die Zeile nicht um (stehen nebeneinander)
- Beispiele: `<a>`, `<span>`, `<img>`, `<strong>`
- Breiten- und Höhenangaben funktionieren hier **nicht**!

DAS BOX-MODELL 📦

- **Content**: Der Inhalt (Text, Bild)
- **Padding**: Der Innenabstand zwischen Inhalt und Rahmen
- **Border**: Der Rahmen um das Element
- **Margin**: Der Außenabstand zu anderen Elementen


Margin box
Border box
Padding box
Content box

BOX-SIZING

- Standardmäßig (`content-box`) vergrößern Padding und Border die Box nach außen.
- Mit `border-box` bleiben die Maße fix, Padding und Border gehen nach innen.

Best Practice:

```
* {
  box-sizing: border-box;
}
```

EINHEITEN IN CSS

ABSOLUTE EINHEITEN

- `px`: Pixel (festgelegte Größe auf dem Bildschirm)

RELATIVE EINHEITEN

- `em`: Relativ zur Schriftgröße des Elternelements
- `rem`: Relativ zur Schriftgröße des Wurzel-Elements (`<html>`)
- `%`: Relativ zum Elternelement

Tipp: Nutze `rem` für Abstände und Schriftgrößen, um die Seite besser skalierbar zu machen.

LAYOUT-TECHNIKEN (MODERN)

FLEXBOX (1-DIMENSIONAL)

Ideal, um Elemente in einer Reihe oder Spalte auszurichten.

```
.container {
  display: flex;
  justify-content: space-between; /* Horizontal verteilen */
  align-items: center;
/* Vertikal zentrieren */
}
```

CSS GRID (2-DIMENSIONAL)

Ideal für komplexe Raster-Layouts (Spalten UND Zeilen).

```
.grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr); /* 3 gleich große Spalten
  gap: 20px;
                /* Abstand zwischen den Zellen */
}
```

RESPONSIVE DESIGN

Webseiten müssen auf Handys und Desktops funktionieren.

MEDIA QUERIES

```
/* Standard für Desktop */
.sidebar { width: 300px; }

/* Anpassung für kleine Bildschirme (max 768px) */
@media (max-width: 768px) {
  .sidebar {
    width: 100%;
    display: none; /* Sidebar auf Handy ausblenden? */
  }
}
```

BROWSER DEV-TOOLS

DEIN BESTER FREUND 👬

- Rechtsklick ➞ Untersuchen
- Du kannst HTML/CSS live im Browser ändern und testen
- Perfekt um das Box-Modell zu visualisieren

FINALE ÜBUNG: LAYOUT & RESPONSIVE

🚀

Aufgabe: Mache deine Landingpage fit für die Zukunft.

- Nutze **Flexbox**, um das Menü oder den Header auszurichten
- Erstelle ein **Grid** für die Produktvorteile
- Nutze **Media Queries**, um das Layout auf schmalen Bildschirmen anzupassen (z.B. Spalten untereinander stapeln)
- **Zusatz:** Style eine Tabelle mit dem "Zebra-Look"